



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Filière : Génie Informatique

Rapport de Projet de Fin d'Études

MoviesApp Azizy

Application Android de Découverte de Films

Intégration TMDB · ML Kit · Google Maps · FileProvider

Réalisé par : Azizy

Encadrant : *À compléter*

Module : Développement Mobile Android

Année académique : 2025 – 2026

Membre de **Honoris United Universities**

Année Universitaire 2025 – 2026

Dédicace

À ma famille, pour son soutien indéfectible tout au long de ce parcours.

À mes enseignants, pour leurs précieux conseils et leur accompagnement bienveillant.

À tous ceux qui croient en la puissance de la technologie au service de l'utilisateur.

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant pour ses orientations judicieuses et sa disponibilité tout au long de ce projet. Mes remerciements s'adressent également à l'ensemble du corps enseignant pour la qualité de la formation dispensée.

Je remercie chaleureusement mes collègues pour les échanges constructifs et le soutien mutuel lors des phases les plus exigeantes du développement.

Enfin, je suis reconnaissant envers les communautés open-source et les équipes de Google, The Movie Database (TMDB) et Anthropic pour les ressources et APIs de grande qualité mises à disposition des développeurs.

Résumé

Mots-clés : Android, Java, TMDB, Volley, Glide, ML Kit, Google Maps, FileProvider, ExifInterface, Géolocalisation.

Ce rapport présente le développement de **MoviesApp Azizy**, une application mobile Android conçue pour offrir une expérience de découverte de films riche et intelligente. L'application s'appuie sur l'API The Movie Database (TMDB) pour fournir un catalogue exhaustif de films, et intègre plusieurs services Google Play pour enrichir les fonctionnalités : intelligence artificielle pour la validation des photos de profil, géolocalisation précise pour la recherche de cinémas à proximité, et gestion avancée de la caméra pour des profils en haute résolution.

L'architecture est fondée sur le SDK Android avec Java, la bibliothèque Volley pour les communications réseau, et Glide pour l'affichage optimisé des médias. Les résultats montrent une application fluide, sécurisée et géolocalisée, répondant aux exigences modernes du développement mobile.

Abstract

Key Words : Android, Java, TMDB, Volley, Glide, ML Kit, Google Maps, File-Provider, ExifInterface, Geolocation.

This report presents the development of **MoviesApp Azizy**, an Android mobile application designed to deliver a rich and intelligent movie discovery experience. The application leverages The Movie Database (TMDB) API to provide a comprehensive film catalogue, and integrates several Google Play services : artificial intelligence for profile photo validation, precise geolocation for finding nearby cinemas, and advanced camera management for high-resolution profile pictures.

The architecture is based on the Android SDK with Java, the Volley library for network communications, and Glide for optimised media display. Results demonstrate a fluid, secure, and geolocated application meeting modern mobile development standards.

Table des figures

Liste des tableaux

1.1	Planification des sprints du projet	12
3.1	Stack technique de MoviesApp Azizy	15
3.2	Environnement de développement	19

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
SDK	Software Development Kit
TMDB	The Movie Database
UI	User Interface
URI	Uniform Resource Identifier
UML	Unified Modeling Language
XML	eXtensible Markup Language

Table des matières

Dédicace	1
Remerciements	2
Résumé	3
Abstract	4
Liste des Figures	5
Liste des Tableaux	6
Liste des Abréviations	7
Introduction Générale	10
1 Contexte Général du Projet	11
1.1 Problématique	11
1.2 Solution Proposée	11
1.3 Méthodologie de Travail et Planification	12
2 Analyse et Conception	13
2.1 Étude Fonctionnelle	13
2.1.1 Analyse des Besoins Fonctionnels	13
2.1.2 Analyse des Besoins Non Fonctionnels	13
2.2 Étude Conceptuelle	14
2.2.1 Règles de Gestion	14
2.2.2 Diagrammes UML	14
3 Étude Technique	15
3.1 Architecture Technique Générale	15
3.2 Technologies Utilisées	15
3.2.1 Volley – Gestion des Requêtes Réseau	15
3.2.2 Glide – Affichage Optimisé des Posters	16

3.2.3	Module de Profil HD – FileProvider et ExifInterface	16
3.2.4	Intelligence Artificielle – ML Kit Face Detection	17
3.2.5	Géolocalisation – FusedLocationProviderClient	18
3.3	Environnement de Développement	19
4	Mise en Œuvre	20
4.1	Objectifs du Projet	20
4.2	Sécurité et Permissions	20
4.2.1	Gestion des Permissions Dynamiques (Android 13+)	20
4.2.2	Sécurisation du Partage de Fichiers – file_paths.xml	21
4.3	Défis Rencontrés et Solutions Adoptées	22
4.3.1	Défi 1 – Qualité de la Caméra	22
4.3.2	Défi 2 – Recherche Contextuelle de Cinémas	22
	Conclusion Générale et Perspectives	23
	Bibliographie	24

Introduction Générale

Contexte du projet

Le cinéma est aujourd'hui l'une des industries culturelles les plus dynamiques au monde. Face à la multiplication des plateformes de streaming et la diversité de l'offre filmique, les utilisateurs ressentent le besoin d'un outil centralisé, intelligent et personnalisé pour explorer et découvrir de nouveaux films. **MoviesApp Azizy** répond précisément à ce besoin en proposant une application Android complète, intégrant des fonctionnalités avancées basées sur les services Google Play.

Problématique

Comment concevoir une application mobile Android alliant découverte de contenu, personnalisation du profil utilisateur et services de géolocalisation, tout en garantissant la qualité des données, la sécurité des échanges et une expérience utilisateur fluide ?

Solution Proposée

- La solution développée repose sur une architecture modulaire combinant :
- L'API TMDB pour l'accès aux données filmographiques ;
 - Google ML Kit pour la validation intelligente des photos de profil ;
 - Le SDK Google Maps et le `FusedLocationProviderClient` pour la géolocalisation ;
 - `FileProvider` et `ExifInterface` pour la gestion avancée de la caméra.

Architecture du Chapitre

Ce rapport est organisé en quatre chapitres : le contexte général du projet, l'analyse et la conception, l'étude technique, puis la mise en œuvre. Une conclusion générale et des perspectives clôturent le document.

Chapitre 1

Contexte Général du Projet

Introduction

Ce chapitre présente le cadre général dans lequel s'inscrit le projet MoviesApp Azizy, en définissant la problématique, la solution retenue et la méthodologie de travail adoptée.

1.1 Problématique

Les applications de découverte de films existantes souffrent de plusieurs limitations : manque de personnalisation du profil, absence d'intégration géographique pour localiser les salles de cinéma, et qualité médiocre des photos de profil due à l'utilisation de miniatures compressées. De plus, aucune validation intelligente du contenu visuel n'est généralement mise en place, exposant les plateformes à des abus.

1.2 Solution Proposée

MoviesApp Azizy propose une réponse globale à ces limitations grâce à :

Périmètre de la solution

- **Découverte de films** via l'intégration complète de l'API TMDB ;
- **Profil HD** avec capture de photos en haute résolution via FileProvider ;
- **IA embarquée** pour valider la présence d'un visage humain sur la photo de profil ;
- **Géolocalisation contextuelle** pour rechercher les cinémas les plus proches.

1.3 Méthodologie de Travail et Planification

Le projet a été conduit selon une approche agile adaptée, décomposée en sprints bihebdomadaires :

TABLE 1.1 – Planification des sprints du projet

Sprint	Objectif	Durée
1	Initialisation, intégration TMDB, affichage liste de films	2 semaines
2	Module profil, intégration Glide, interface utilisateur	2 semaines
3	Module caméra HD (FileProvider + ExifInterface)	2 semaines
4	Intégration ML Kit (détection faciale)	2 semaines
5	Géolocalisation, Google Maps, recherche cinémas	2 semaines
6	Tests, optimisations, sécurité et permissions	2 semaines

Conclusion

Ce chapitre a posé les bases conceptuelles du projet. Le suivant détaillera l'analyse fonctionnelle et la conception architecturale de l'application.

Chapitre 2

Analyse et Conception

Introduction

Ce chapitre présente l'étude fonctionnelle et conceptuelle de MoviesApp Azizy, en identifiant les besoins des utilisateurs et en modélisant l'architecture logicielle.

2.1 Étude Fonctionnelle

2.1.1 Analyse des Besoins Fonctionnels

Les besoins fonctionnels principaux de l'application sont les suivants :

- **BF01** – Afficher la liste des films populaires et en cours de diffusion ;
- **BF02** – Rechercher un film par titre ou genre ;
- **BF03** – Consulter la fiche détaillée d'un film (synopsis, note, casting) ;
- **BF04** – Gérer un profil utilisateur avec photo en haute résolution ;
- **BF05** – Valider automatiquement la photo de profil par détection faciale ;
- **BF06** – Localiser les cinémas proches de la position GPS de l'utilisateur ;
- **BF07** – Ouvrir Google Maps avec un itinéraire vers le cinéma sélectionné.

2.1.2 Analyse des Besoins Non Fonctionnels

- **Performance** : Temps de chargement des affiches inférieur à 1,5 secondes ;
- **Sécurité** : Partage de fichiers sécurisé via FileProvider ;
- **Compatibilité** : Android 8.0 (API 26) et supérieur, optimisé pour Android 13+ ;
- **Maintenabilité** : Architecture MVC favorisant la séparation des responsabilités.

2.2 Étude Conceptuelle

2.2.1 Règles de Gestion

- Une photo de profil n'est acceptée que si ML Kit détecte au moins un visage humain ;
- La recherche de cinémas utilise obligatoirement la position GPS réelle de l'utilisateur ;
- Les images de films sont chargées exclusivement via l'URL fournie par l'API TMDB ;
- Les permissions Caméra et Localisation sont demandées dynamiquement à l'exécution.

2.2.2 Diagrammes UML

Diagramme des Cas d'Utilisation

Les acteurs principaux sont l'*Utilisateur* et les services externes (*API TMDB*, *Google Play Services*). Les cas d'utilisation couvrent : consulter des films, gérer son profil, valider sa photo et rechercher des cinémas.

Diagramme de Classes Simplifié

Les classes principales sont : *Movie*, *UserProfile*, *ApiManager* (singleton Volley), *CameraManager*, *LocationManager* et *FaceDetector*.

Conclusion

L'analyse fonctionnelle et la modélisation UML ont permis de définir précisément le périmètre applicatif. Le chapitre suivant détaille les choix techniques retenus.

Chapitre 3

Étude Technique

Introduction

Ce chapitre présente l'architecture technique de MoviesApp Azizy, les technologies utilisées et les détails d'implémentation des fonctionnalités avancées.

3.1 Architecture Technique Générale

TABLE 3.1 – Stack technique de MoviesApp Azizy

Composant	Technologie / Bibliothèque
Langage	Java (Android SDK, API 26+)
Gestion réseau	Volley (requêtes JSON asynchrones)
Affichage d'images	Glide 4.x (cache mémoire et disque)
Données films	API TMDb (api.themoviedb.org/3/)
Intelligence artificielle	Google ML Kit – Face Detection
Géolocalisation	FusedLocationProviderClient (Google Play)
Cartographie	Google Maps SDK for Android
Caméra HD	FileProvider + Intent ACTION_IMAGE_CAPTURE
Orientation image	ExifInterface (rotation automatique)
Sécurité fichiers	FileProvider (<code>file_paths.xml</code>)

3.2 Technologies Utilisées

3.2.1 Volley – Gestion des Requêtes Réseau

Volley est la bibliothèque officielle Google pour les requêtes HTTP asynchrones sur Android. Elle gère automatiquement la file d'attente, le cache et les erreurs réseau.


```
1 RequestQueue queue = Volley.newRequestQueue(context);
2 String url = "https://api.themoviedb.org/3/movie/popular"
3           + "?api_key=" + API_KEY + "&language=fr-FR&page=1"
4           ;
5 JSONObjectRequest request = new JSONObjectRequest(
6     Request.Method.GET, url, null,
7     response -> {
8         JSONArray results = response.getJSONArray("results");
9         // Traitement de la liste de films
10    },
11    error -> Log.e("TMDB", "Erreur: " + error.getMessage())
12 );
13 queue.add(request);
```

Listing 3.1 – Requête TMDB avec Volley

3.2.2 Glide – Affichage Optimisé des Posters

Glide gère le chargement, le redimensionnement et la mise en cache des images de films, garantissant une expérience fluide même sur connexions lentes.

```
1 Glide.with(context)
2     .load("https://image.tmdb.org/t/p/w500" + movie.
3         getPosterPath())
4     .placeholder(R.drawable.ic_movie_placeholder)
5     .error(R.drawable.ic_error)
6     .centerCrop()
7     .into(imageView);
```

Listing 3.2 – Chargement d'un poster avec Glide

3.2.3 Module de Profil HD – FileProvider et ExifInterface

L'utilisation directe de `ACTION_IMAGE_CAPTURE` sans `FileProvider` produit des miniatures de basse résolution (environ 150×150 px). La solution retenue utilise `FileProvider` pour forcer la capture en pleine résolution.

```
1 // Création du fichier image dans le repertoire priv
2 File photoFile = new File(context.getFilesDir(), "profile_hd.
3     jpg");
4 Uri photoUri = FileProvider.getUriForFile(
```

```
4     context ,
5     context.getPackageName() + ".fileprovider",
6     photoFile
7 );
8
9 Intent cameraIntent = new Intent(MediaStore.
    ACTION_IMAGE_CAPTURE);
10 cameraIntent.putExtra(MediaStore.EXTRA_OUTPUT, photoUri);
11 startActivityForResult(cameraIntent, REQUEST_CAMERA);
```

Listing 3.3 – Capture photo HD via FileProvider

La rotation automatique est ensuite gérée par ExifInterface :

```
1 ExifInterface exif = new ExifInterface(photoFile.
    getAbsolutePath());
2 int orientation = exif.getAttributeInt(
3     ExifInterface.TAG_ORIENTATION,
4     ExifInterface.ORIENTATION_NORMAL
5 );
6 Matrix matrix = new Matrix();
7 switch (orientation) {
8     case ExifInterface.ORIENTATION_ROTATE_90: matrix.
9         postRotate(90); break;
10    case ExifInterface.ORIENTATION_ROTATE_180: matrix.
11        postRotate(180); break;
12    case ExifInterface.ORIENTATION_ROTATE_270: matrix.
13        postRotate(270); break;
14 }
15 Bitmap correctedBitmap = Bitmap.createBitmap(
16     originalBitmap, 0, 0,
17     originalBitmap.getWidth(), originalBitmap.getHeight(),
18     matrix, true
19 );
```

Listing 3.4 – Correction de rotation avec ExifInterface

3.2.4 Intelligence Artificielle – ML Kit Face Detection

Google ML Kit est intégré pour valider que la photo de profil contient bien un visage humain, renforçant ainsi la qualité et la cohérence des profils.

```
1 FaceDetectorOptions options = new FaceDetectorOptions.Builder
    ()
```

```
2      .setPerformanceMode(FaceDetectorOptions.  
        PERFORMANCE_MODE_FAST)  
3      .setMinFaceSize(0.15f)  
4      .build();  
5  
6  FaceDetector detector = FaceDetection.getClient(options);  
7  InputImage image = InputImage.fromBitmap(profileBitmap, 0);  
8  
9  detector.process(image)  
10     .addOnSuccessListener(faces -> {  
11         if (faces.isEmpty()) {  
12             Toast.makeText(context,  
13                 "Aucun visage détecté. Veuillez reprendre la photo.",  
14                 Toast.LENGTH_LONG).show();  
15         } else {  
16             saveProfilePhoto(); // Validation réussie  
17         }  
18     })  
19     .addOnFailureListener(e -> Log.e("MLKit", e.getMessage()))  
    );
```

Listing 3.5 – Détection faciale avec ML Kit

3.2.5 Géolocalisation – FusedLocationProviderClient

Le `FusedLocationProviderClient` fusionne GPS, Wi-Fi et réseau cellulaire pour obtenir la position la plus précise avec une consommation d'énergie optimisée.

```
1  FusedLocationProviderClient fusedClient =  
2      LocationServices.getFusedLocationProviderClient(this);  
3  
4  if (ActivityCompat.checkSelfPermission(this,  
5      Manifest.permission.ACCESS_FINE_LOCATION)  
6      == PackageManager.PERMISSION_GRANTED) {  
7  
8      fusedClient.getLastLocation()  
9          .addOnSuccessListener(location -> {  
10             if (location != null) {  
11                 double lat = location.getLatitude();  
12                 double lon = location.getLongitude();  
13                 searchNearbyCinemas(lat, lon);  
            }  
        })  
    }  
}
```

```
14         }
15     });
16 }
```

Listing 3.6 – Obtention de la position GPS précise

La recherche de cinémas à proximité utilise un Intent géographique :

```
1 Uri mapsUri = Uri.parse(
2     "geo:" + lat + "," + lon + "?q=cinema"
3 );
4 Intent mapsIntent = new Intent(Intent.ACTION_VIEW, mapsUri);
5 mapsIntent.setPackage("com.google.android.apps.maps");
6 startActivity(mapIntent);
```

Listing 3.7 – Ouverture Google Maps pour les cinémas proches

3.3 Environnement de Développement

TABLE 3.2 – Environnement de développement

Outil	Version / Détail
IDE	Android Studio Hedgehog 2023.1.1
Langage	Java 17
Gradle	8.x
Android SDK cible	API 34 (Android 14)
SDK minimum	API 26 (Android 8.0)
Émulateur de test	Pixel 7 API 34 / Appareil physique
Contrôle de version	Git / GitHub

Conclusion

L'étude technique a démontré la richesse de l'écosystème Android et Google Play Services. Le chapitre suivant présentera la mise en œuvre concrète et les résultats obtenus.

Chapitre 4

Mise en Œuvre

Introduction

Ce chapitre décrit la mise en œuvre effective de MoviesApp Azizy, couvrant la sécurité, les permissions, les défis rencontrés et les solutions adoptées.

4.1 Objectifs du Projet

MoviesApp Azizy vise à :

1. Offrir une **expérience utilisateur fluide** grâce à un chargement asynchrone ;
2. Garantir la **sécurité des données** via FileProvider et permissions dynamiques ;
3. Fournir des services **géolocalisés et contextuels** pour les cinémas proches ;
4. Intégrer l'**intelligence artificielle** de manière transparente pour l'utilisateur.

4.2 Sécurité et Permissions

4.2.1 Gestion des Permissions Dynamiques (Android 13+)

Depuis Android 6.0 (API 23), les permissions sensibles doivent être demandées à l'exécution. Pour Android 13+, cela inclut également `READ_MEDIA_IMAGES`.

```
1 private void requestPermissions() {
2     String[] permissions = {
3         Manifest.permission.CAMERA,
4         Manifest.permission.ACCESS_FINE_LOCATION,
5         Manifest.permission.ACCESS_COARSE_LOCATION
6     };
7
8     if (!hasPermissions(permissions)) {
```

```

9         ActivityCompat.requestPermissions(this, permissions,
            REQ_CODE);
10     }
11 }
12
13 @Override
14 public void onRequestPermissionsResult(int requestCode,
15     String[] permissions, int[] grantResults) {
16     super.onRequestPermissionsResult(requestCode, permissions
17         , grantResults);
18     if (requestCode == REQ_CODE) {
19         for (int result : grantResults) {
20             if (result != PackageManager.PERMISSION_GRANTED)
21             {
22                 showPermissionRationale();
23                 return;
24             }
25         }
26         initializeFeatures(); // Toutes les permissions
            accord es
27     }
28 }

```

Listing 4.1 – Demande de permissions dynamiques

4.2.2 Sécurisation du Partage de Fichiers – file_paths.xml

FileProvider nécessite une déclaration des chemins autorisés dans `res/xml/file_paths.xml` :

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <paths>
3     <!-- R pertoire interne priv de l'application -->
4     <files-path name="internal_files" path="." />
5     <!-- R pertoire de cache pour les fichiers temporaires -->
6     <cache-path name="cache_files" path="." />
7     <!-- R pertoire externe priv (si n cessaire) -->
8     <external-files-path name="external_files" path="." />
9 </paths>

```

Listing 4.2 – Configuration file_paths.xml

```

1 <provider

```

```
2     android:name="androidx.core.content.FileProvider"
3     android:authorities="${applicationId}.fileprovider"
4     android:exported="false"
5     android:grantUriPermissions="true">
6     <meta-data
7         android:name="android.support.FILE_PROVIDER_PATHS"
8         android:resource="@xml/file_paths" />
9 </provider>
```

Listing 4.3 – Déclaration du FileProvider dans AndroidManifest.xml

4.3 Défis Rencontrés et Solutions Adoptées

4.3.1 Défi 1 – Qualité de la Caméra

Problème : L'utilisation de `ACTION_IMAGE_CAPTURE` sans paramètre `EXTRA_OUTPUT` retourne uniquement une miniature JPEG dégradée dans le bundle `onActivityResult`, insuffisante pour un profil utilisateur professionnel.

Solution : L'utilisation de **URI de contenu via FileProvider** force l'appareil photo à sauvegarder l'image en pleine résolution dans un fichier dédié. L'URI est passé en `EXTRA_OUTPUT`, garantissant une qualité maximale. `ExifInterface` corrige ensuite automatiquement l'orientation selon les métadonnées EXIF encodées par le capteur lors de la prise de vue.

Résultat

Résolution de la photo de profil passée de $\approx 150 \times 150$ px à la résolution native du capteur (ex. 3024×4032 px).

4.3.2 Défi 2 – Recherche Contextuelle de Cinémas

Problème : Une recherche de cinémas générique (sans position) retourne des résultats non pertinents géographiquement, dégradant l'expérience utilisateur.

Solution : Le `FusedLocationProviderClient` obtient la position GPS précise de l'utilisateur en temps réel. La requête géographique est construite dynamiquement avec les coordonnées réelles (`geo:lat,lon?q=cinema`), garantissant que Google Maps affiche uniquement les cinémas à proximité immédiate. Cette approche contextuelle améliore significativement la pertinence des résultats.

Conclusion Générale et Perspectives

Bilan du Projet

Ce projet a permis de concevoir et développer **MoviesApp Azizy**, une application Android complète qui dépasse le simple catalogue de films pour offrir une expérience utilisateur enrichie et intelligente. Les principaux apports techniques résident dans :

- L'intégration maîtrisée de l'**API TMDb** pour des données filmographiques en temps réel ;
- Un **module caméra HD** robuste grâce à FileProvider et ExifInterface, résolvant les problèmes de qualité natifs d'Android ;
- Une **validation IA** transparente des photos de profil via Google ML Kit ;
- Une **géolocalisation contextuelle** précise, couplée à Google Maps, offrant une valeur ajoutée concrète à l'utilisateur.

Ce projet a démontré la complexité et la richesse de l'**écosystème Google Play Services**, dont l'intégration requiert une maîtrise fine des APIs, des permissions Android 13+, et des bonnes pratiques de sécurité.

Perspectives et Évolutions

Plusieurs pistes d'évolution sont envisagées pour les versions futures :

1. **Système de Favoris** : Permettre à l'utilisateur de sauvegarder des films en favoris, avec synchronisation cloud via Firebase Firestore ;
2. **Notifications Push** : Alerter l'utilisateur lors de la sortie d'un film attendu via Firebase Cloud Messaging (FCM) ;
3. **Mode Hors-ligne** : Mise en cache locale des données via Room Database pour un accès sans connexion ;
4. **Recommandations Personnalisées** : Moteur de recommandation basé sur l'historique de consultation de l'utilisateur ;
5. **Réalité Augmentée** : Intégration d'ARCore pour visualiser des informations contextuelles sur les affiches de films détectées par la caméra.

Bibliographie

- [1] Google LLC. *Android Developer Documentation*.
<https://developer.android.com/docs>
- [2] Google LLC. *ML Kit – Face Detection*.
<https://developers.google.com/ml-kit/vision/face-detection>
- [3] Google LLC. *Maps SDK for Android*.
<https://developers.google.com/maps/documentation/android-sdk>
- [4] Google LLC. *Fused Location Provider API*.
<https://developers.google.com/location-context/fused-location-provider>
- [5] The Movie Database. *TMDB API Documentation*.
<https://developers.themoviedb.org/3>
- [6] Bumptech. *Glide Image Loading Library*.
<https://bintray.com/bumptech/glide>
- [7] Google LLC. *Volley – Networking for Android*.
<https://google.github.io/volley/>
- [8] AndroidX Core. *FileProvider Documentation*.
<https://developer.android.com/reference/androidx/core/content/FileProvider>